# ECE 5730 Lab 3 Report: PID Control of a 1D Helicopter

Amber Li | Ivan Huang | Zarif Karim

10.27.2026

## Introduction

We built and controlled a one-degree-of-freedom "helicopter": a lever arm hinged at one end with a ducted motor at the other. An MPU-6050 IMU on the arm reports acceleration and angular rate over I2C. A Raspberry Pi Pico 2 (RP2350) fuses them with a complementary filter to estimate the arm angle, and a PID controller running at exactly 1 kHz sets the motor's PWM duty cycle through an optically isolated MOSFET driver.

The target angle and gains are set over a serial terminal and take effect immediately. A VGA display plots the measured angle, the target and the low-passed motor command in real time. A pushbutton runs the demo: the arm hangs down while the button is held, then steps to 0°, +30°, −30° and 0° at 5 s intervals.

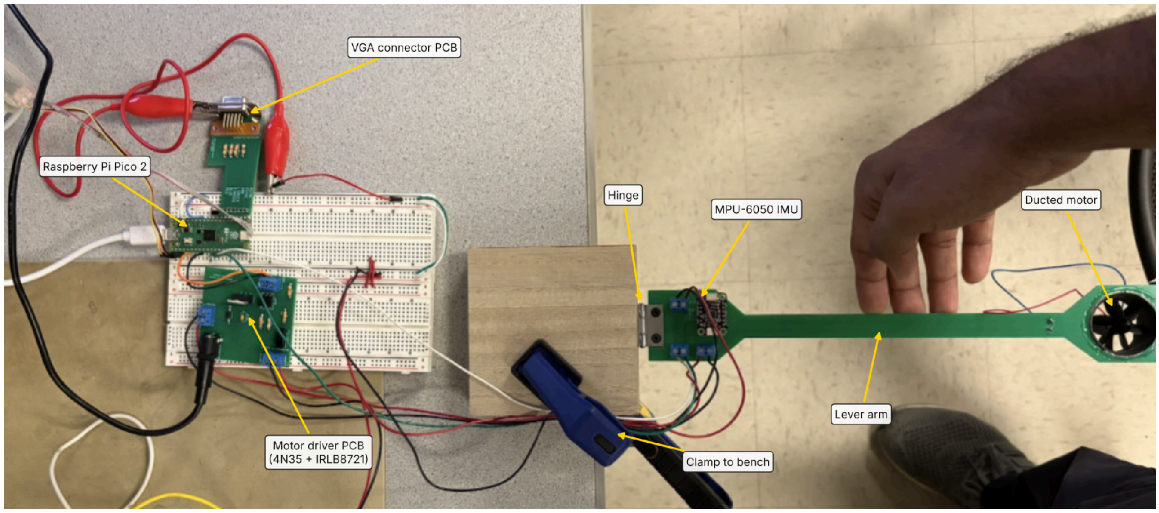



*Figure 1: The rig from above. The arm pivots on a hinge clamped to the bench; the IMU sits beside the hinge and the ducted motor at the far end.*

## Mathematical Background

### Complementary Filter

The accelerometer measures the direction of gravity, so it gives an absolute angle that never drifts, but motor vibration and arm acceleration make it noisy. The gyroscope measures angular rate; integrating it gives a smooth angle that slowly drifts with the gyro's bias (Figure 2).

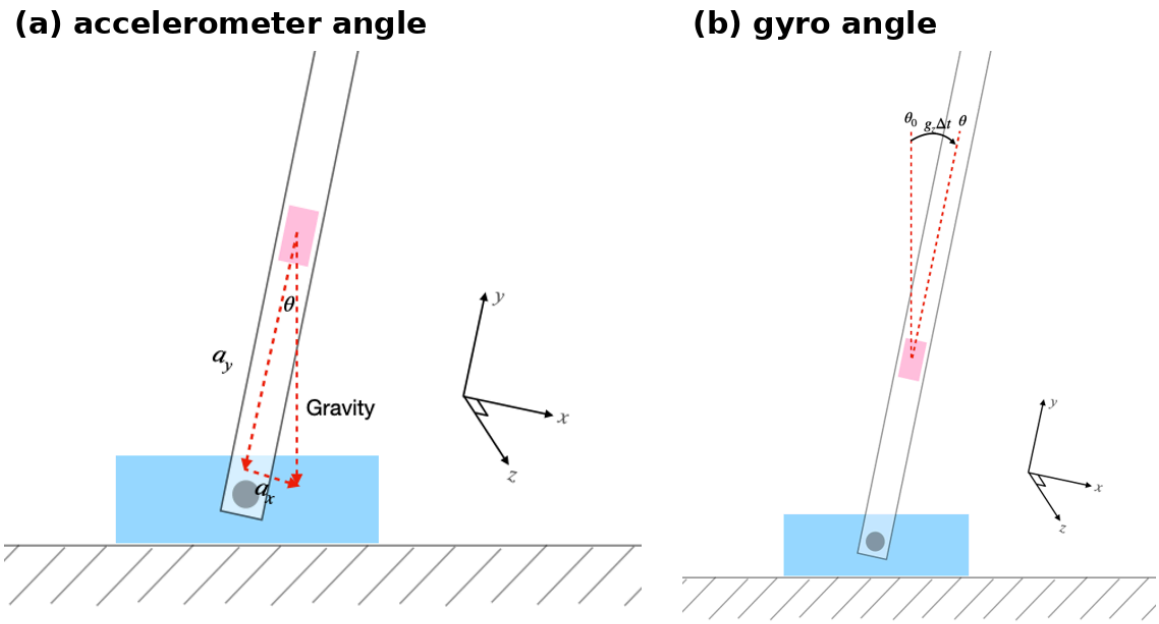



*Figure 2: (a) The accelerometer angle comes from the direction of gravity. (b) The gyro angle adds rate × Δt each step. From the course notes, which label the axes a_y and g_z; on our board the matching axes are a_z and gyro y.*

Each 1 ms step, we add the gyro's angle change to the previous estimate and nudge the result toward the accelerometer angle (Figure 3):

$$\theta_n = 0.999\,\left(\theta_{n-1} + \dot{\theta}_{gyro}\,\Delta t\right) + 0.001\,\theta_{accel}$$

The gyro path acts as a high-pass filter and the accelerometer path as a low-pass, crossing over at roughly Δt / (1 − 0.999) ≈ 1 s. More accelerometer weight means less drift but more noise. The notes suggest starting at 0.01; we used 0.001 to keep more motor vibration out of the estimate.

Because the arm swings through large angles, we use atan2 instead of the small-angle approximation:

$$\theta_{accel} = \operatorname{atan2}(-a_x,\ a_z) \cdot \frac{180}{\pi}$$

0° is horizontal, −90° is hanging down (the handout's −1.57 rad start) and positive is above horizontal. Before atan2, a_x and a_z pass through a first-order IIR low-pass, y += (x − y) / 16, with a 16 ms time constant.

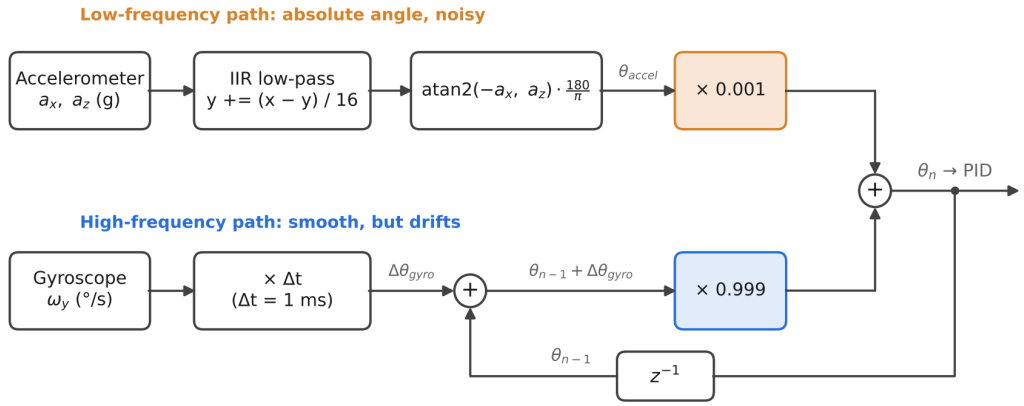



*Figure 3: Our complementary filter.*

## PID Control

Open-loop control (a lookup table from angle to duty cycle) fails because the thrust needed changes with battery voltage, motor temperature and air density, and nothing corrects a bump. We close the loop instead (Figure 4).

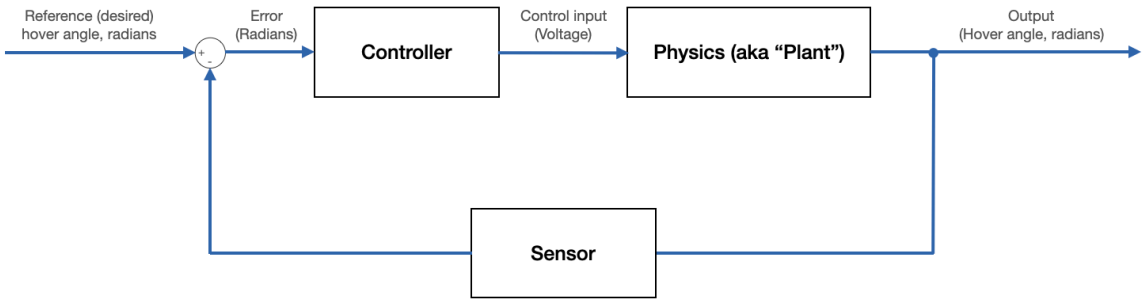



*Figure 4: Closed-loop control. From the course PID notes.*

$$u(t) = K_p\, e(t) + K_i \int e(t)\, dt + K_d \frac{de(t)}{dt}, \qquad e = \theta_{desired} - \theta_{measured}$$

- **Proportional** scales the current error (Figure 5a). Alone it leaves a steady-state error, and too much Kp makes the arm oscillate.
- **Integral** scales the accumulated error (Figure 5b). It removes steady-state error and, once settled, supplies the duty that holds the arm up. It adds phase lag, so Ki stays small.
- **Derivative** scales the error's rate of change. It opposes fast approaches, which adds damping.

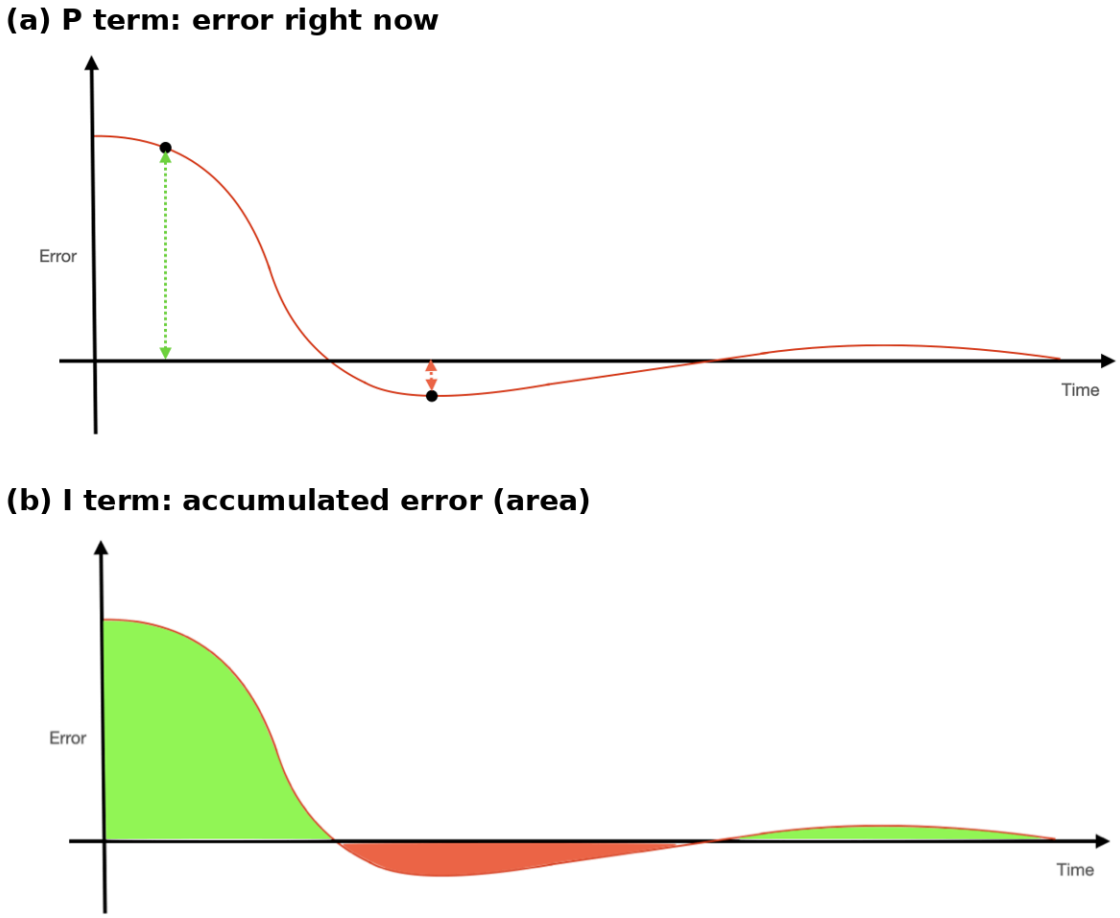



*Figure 5: (a) P acts on the error at this instant. (b) I acts on the area under the error curve, positive and negative. From the course PID notes.*

The target is constant between commands, so the error's derivative is minus the arm's angular rate:

$$\frac{de}{dt} = -\dot{\theta} = -\omega_{gyro}$$

We feed the gyro reading straight into the D term. Differencing the angle estimate would amplify noise, and its one-sample change at 1 kHz is smaller than the noise.

### Modeling the Plant

The analytical PID notes model the system as a motor (voltage to rotor speed ω) in series with the arm (ω to angle). Here θ is measured from hanging down, as in the notes (Figure 6); our code's angle is θ − 90°.

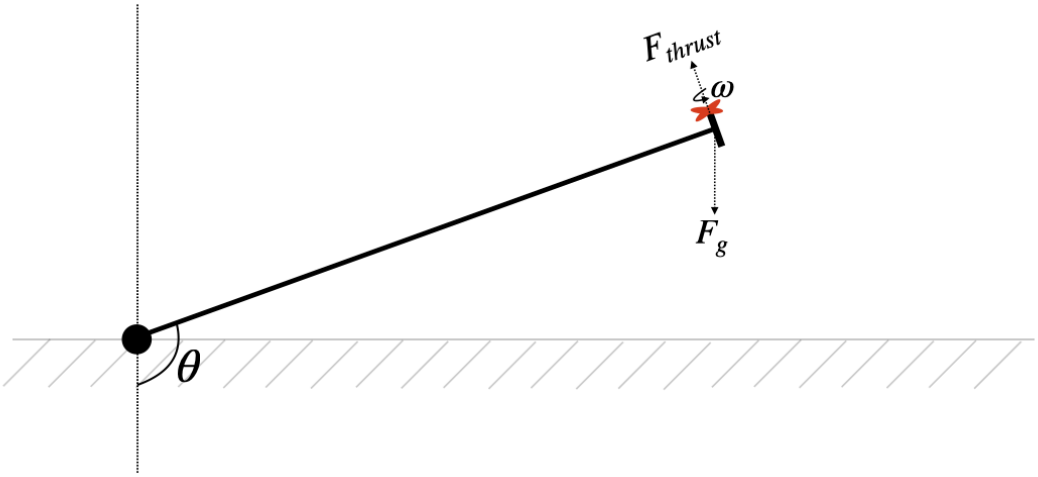



*Figure 6: Thrust and gravity on the arm. From the course PID notes.*

**Motor.** An armature with resistance R, inductance L and back-EMF constant K, driving inertia J with drag b, gives a fast two-pole low-pass. The arm is much slower, so it dominates the loop.

$$\frac{\Omega(s)}{V(s)} = \frac{K}{(Js+b)(Ls+R)+K^2}$$

**Arm.** With thrust K_T·ω², motor mass m at distance L and friction K_f, the torque balance and its linearization about $\theta_0$ and $\omega_0$ are:

$$K_T\,\omega^2 L - mgL\sin\theta - K_f\,\dot\theta = mL^2\,\ddot\theta$$

$$\Delta\ddot\theta = \frac{2K_T\omega_0}{mL}\,\Delta\omega - \frac{g}{L}\cos\theta_0\,\Delta\theta - \frac{K_f}{mL^2}\,\Delta\dot\theta$$

About hanging down, this is a lightly damped resonance at √(g/L): 5.7 rad/s for the notes' 0.3 m arm, [TODO: L = __ m → __ rad/s] for ours. The gravity stiffness (g/L)·cos $\theta_0$ is positive below horizontal, zero at horizontal and negative above it, so above horizontal the arm is unstable on its own.

**Effect of each gain.** The combined open-loop model is type 0 (no free integrator).

- **Kp** raises the magnitude plot, shrinking both the steady-state error and the gain margin. At 0 dB with −180° of phase, the arm oscillates without decaying (Figure 12).
- **Kd** adds up to +90° of phase near the resonance, restoring margin.
- **Ki** adds a free integrator, so steady-state error goes to zero, but it costs 90° of phase.

## Practical PID Problems

Table 1 lists the failure modes the course notes warn about and how we handled each. Figure 7 shows the worst one, integrator windup.

| Problem | Our fix |
|---|---|
| Integrator windup: error keeps accumulating while the arm is held or the motor saturates | Clamp i_term to [0, i_max], below DUTY_MAX; clear it whenever closed loop is off |
| Derivative noise: differentiating multiplies noise by its frequency | D term uses the gyro rate; no numerical derivative |
| Tiny derivative signal: at 1 kHz the per-sample angle change is below the noise | Same: the gyro measures rate directly |
| Accelerometer noise from motor vibration | IIR low-pass before atan2, then 0.999 weight on the gyro |
| Noisy control signal on the display | Plot motor_disp, a 16-sample low-pass of the command |

*Table 1: PID implementation problems and our fixes.*

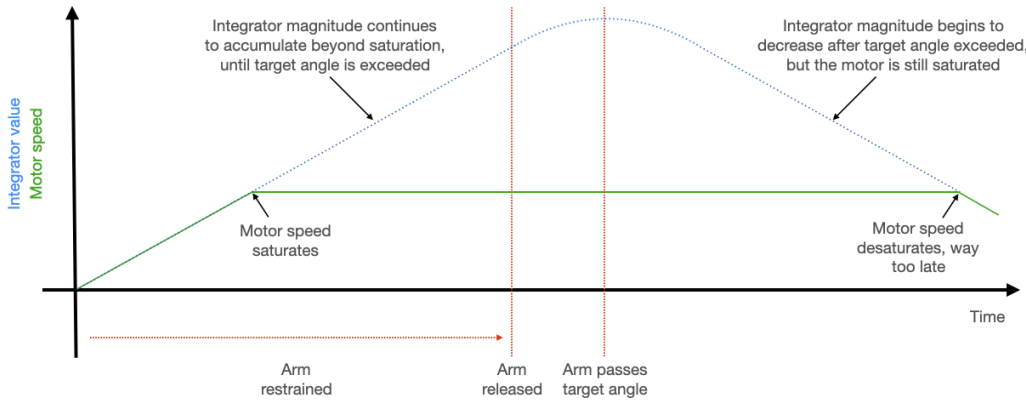



*Figure 7: Integrator windup. The integrator keeps growing past saturation, so the motor stays saturated long after the arm passes the target. From the course PID notes.*

## Fixed Point Arithmetic

All ISR math is fixed point except atan2, which has no cheap fixed-point version. fix15 is a signed 15.16 number in a 32-bit int (Figure 8): range about ±32,768, resolution $1.5 \times 10^{-5}$.

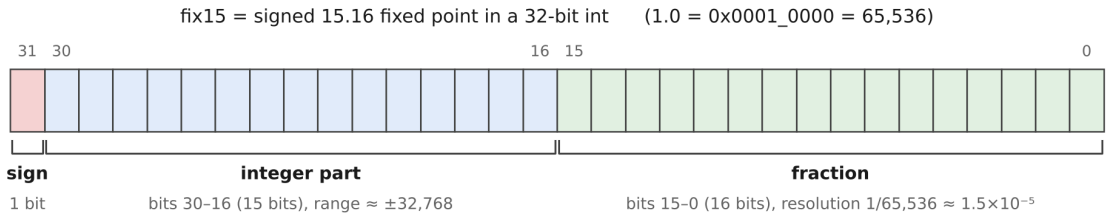


*Figure 8: fix15 bit layout.*

multfix15 multiplies in 64 bits and shifts right by 16. In the PID code we shift Kp × error right by 32 instead, which gives an integer duty count directly. Constants are pre-scaled (0.001 → 65, 0.999 → 65,470), and the IMU driver converts raw readings:

- Accelerometer (±2 g): 16,384 counts per g, shifted left 2 → 65,536 = 1.0 g.
- Gyroscope (±250 °/s): 131 counts per °/s, × 500 >> 16 ≈ raw / 131.

## Design and Testing Methods

We built and tested in stages: the motor circuit with a function generator, then open-loop PWM, the IMU and complementary filter on VGA, P, D, I, and finally the demo button.

### Motor Circuit and PWM

The motor runs from its own 5 V supply through an IRLB8721 MOSFET, and the Pico drives the gate through a 4N35 optoisolator. As the handout requires, the circuit is on a PCB (Figure 9).

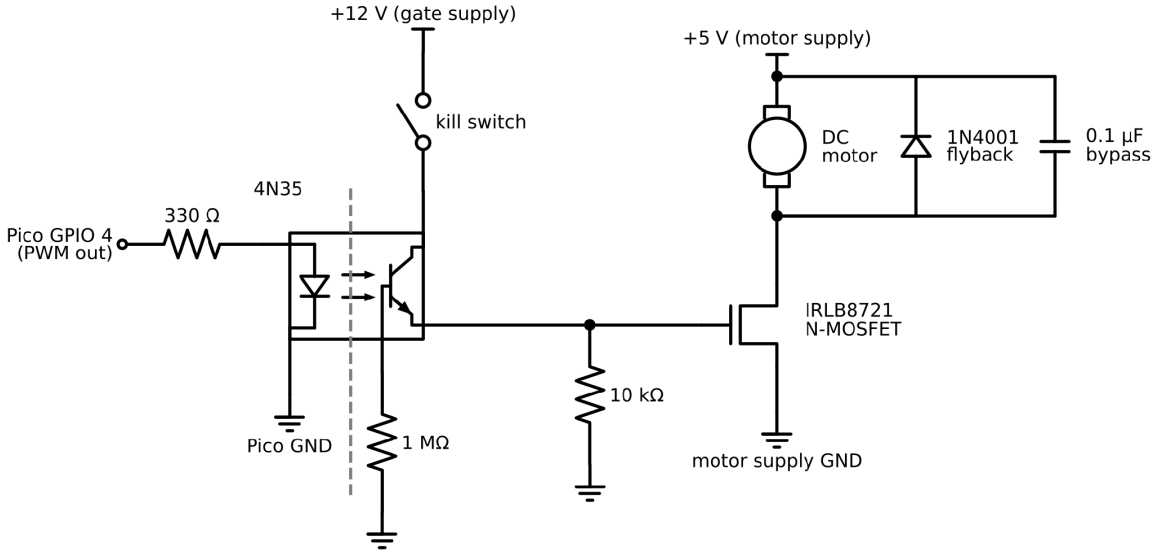



*Figure 9: Motor driver schematic. Only light crosses the dashed isolation barrier.*

- **Optoisolator:** the Pico and motor sides share no ground, so motor noise cannot reach the Pico or IMU. When GPIO 4 is high, the phototransistor pulls the gate toward 12 V and the motor runs; when it is low, the 10 kΩ resistor pulls the gate down. The 1 MΩ base resistor shortens the 4N35's fall time.
- **Flyback diode (1N4001):** switching off the armature's inductance produces $V = L \cdot dI/dt$. The notes' example, 1 mH and 1 A switched off in 10 ns, gives $10^5$ V. The diode gives that current a safe path.
- **Bypass capacitor (0.1 µF ceramic):** shunts brush noise above 100 kHz. It must be ceramic, because electrolytics are too slow.

We first drove the board with a function generator and confirmed that motor speed tracked duty cycle. [TODO: scope screenshot or note.]

PWM is on GPIO 4 (slice 2, channel A):

[image: PWM frequency calculation]

The 4N35's low bandwidth limits the PWM to about this frequency, and the same 1 ms period sets the control rate. The duty cycle runs from 0 to 6000; the controller is capped at DUTY_MAX = 4500 [TODO: why]. Open loop, the arm holds horizontal at [TODO] counts.

### MPU-6050 and I2C

The IMU talks to the Pico over I2C0 at 400 kHz (SDA on GPIO 8, SCL on GPIO 9). The arm pivots about the IMU's y axis, so we use a_x, a_z and gyro y. Table 2 lists the setup registers.

| Register | Value | Effect |
|---|---|---|
| 0x6B PWR_MGMT_1 | 0x00 | Wake from sleep |
| 0x19 SMPLRT_DIV | 7 | 8 kHz / 8 = 1 kHz sample rate |
| 0x1B GYRO_CONFIG | 0x00 | ±250 °/s |
| 0x1C ACCEL_CONFIG | 0x00 | ±2 g |
| 0x37 / 0x38 | 0x10 / 0x01 | Data-ready interrupt (pin unused) |

*Table 2: MPU-6050 configuration registers.*

Each read writes the register address, then reads 6 bytes after a repeated start; the register pointer auto-increments. Every byte takes 9 clocks (8 bits plus ACK), so one read is about 81 clocks, or 0.21 ms at 400 kHz. The two reads use about 0.42 ms of each 1 ms period, which is why the rest of the ISR is fixed point and the drawing runs on core 1.

**Stuck bus.** If the Pico reset mid-transfer, the IMU could hold SDA low, and the blocking writes in mpu6050_reset() hung. At startup we now clock SCL up to nine times before enabling I2C, which lets the IMU finish its byte. [TODO: confirm this matches what you saw.] We also seed both filters with the first reading, so the estimate starts at the true angle.

### PID Interrupt Service Routine

The control loop runs in the PWM wrap interrupt, so it fires exactly once per PWM period with no timer to keep in sync. Each call:

1. Clears the interrupt flag and toggles GPIO 14.
2. Reads the IMU.
3. Low-passes a_x and a_z and computes the accelerometer angle.
4. Updates the complementary filter.
5. Debounces the demo button and steps the demo target.
6. Runs PID if closed loop is on, clamped to 0–4500.
7. Writes the PWM level if it changed.
8. Low-passes the command for display and, every 10th call, stores samples for the plots.

On the scope, GPIO 14 is a 500 Hz square wave with 1 ms halves. [Optional: scope screenshot.]

## PID Implementation

The core of the controller:

```
angle_error = desired_angle - complementary_angle ;
long long p_term = ((long long)Kp) * ((long long)angle_error) ;
long long d_term = ((long long)Kd) * ((long long)gyro[1]) ;
i_term += multfix15(Ki, angle_error) ;
if (i_term > int2fix15(i_max)) i_term = int2fix15(i_max) ;
if (i_term < 0) i_term = 0 ;
int u = (int)((p_term - d_term) >> 32) + fix2int15(i_term) ;
if (u < 0) u = 0 ;
if (u > DUTY_MAX) u = DUTY_MAX ;
```

- Positive error means the arm is below the target. D is subtracted because de/dt = −ω.
- The integrator accumulates Ki × error, already in duty counts, so it fits in 15.16 and changing Ki doesn't make the output jump.
- Anti-windup clamps i_term to [0, i_max] ([TODO: final i_max]). A negative integrator would only add lag, because the motor cannot pull the arm down.
- The output is clamped to 0–4500, and the integrator is cleared whenever closed loop is off.

## VGA Display

The Pico drives the VGA connector through the adapter PCB we soldered in Lab 2 (Figure 10): the two green bits and the red and blue bits each pass through a series resistor, forming a simple resistor DAC.




*Figure 10: Our VGA adapter PCB (photographed during Lab 2).*

We reused the course's double-buffered VGA driver (Figure 11). Three PIO state machines generate hsync, vsync and the 4-bit color output, and a DMA chain streams pixel data to them. At each vsync the

chain swaps buffers and sets a flag, giving the drawing thread about 16.7 ms per frame. If drawing runs late, the half-drawn buffer is shown as a torn frame instead of slowing the display. A frame is 640 × 480 × 4 bits, stored row by row with two pixels per byte (320 bytes per row), so the two buffers use about 300 kB of the RP2350's 520 kB of SRAM.

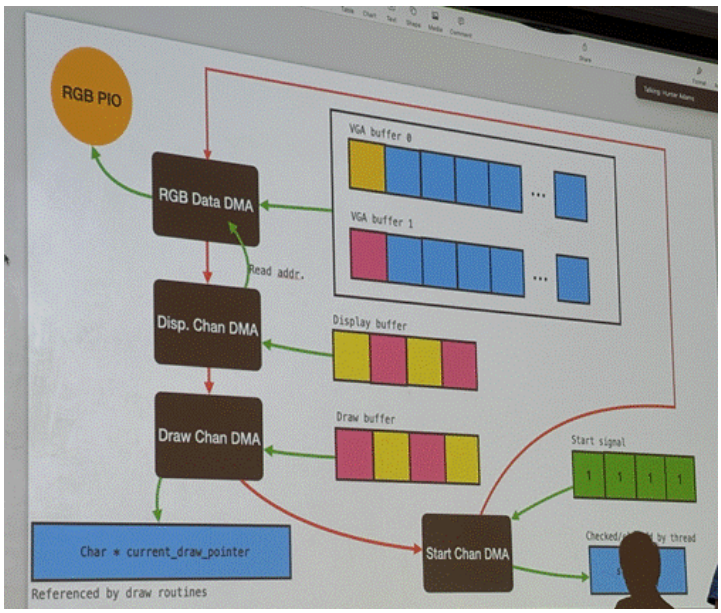



*Figure 11: The VGA driver's DMA chain, from lecture. The RGB data DMA streams the display buffer to the color PIO while the draw routines write to the other buffer.*

Core 1 runs the drawing thread, so drawing never delays the control ISR. The ISR stores one sample every 10 ms in 530-entry buffers, so the plots span 5.3 s (21.2 s with t = 40).

- **Top:** complementary angle (yellow) and target (light blue).
- **Bottom:** low-passed motor command, 0–4500 (orange).
- **Text:** both angle estimates, target, gains, I cap, I term, duty, mode and demo time.

The handout requires the plotted command to be low-passed. Each ISR call updates motor_disp += (control − motor_disp) >> 4, while the motor still gets the raw output. This is a forward-Euler RC filter with Δt / RC = 1/16, a 16 ms time constant; the accelerometer low-pass uses the same update.

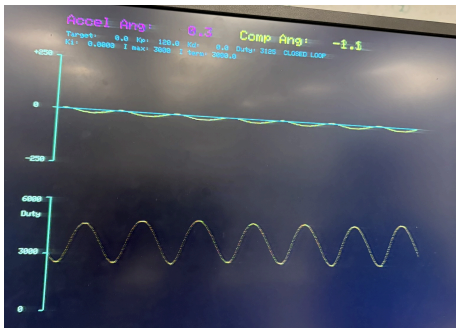



*Figure 12: Early P-only test (Kp = 120, Ki = Kd = 0, target 0°), on an older build that plotted raw duty on a 0–6000 axis. The arm oscillates continuously, about 6.5 cycles in 5.3 s. The integrator was still at its 3,000-count cap from an earlier run.*

### Serial Interface

The serial thread on core 0 waits for a one-letter command, then prompts for a value. New values are written straight to the volatile globals the ISR reads, so they take effect on the next 1 ms cycle.

| Command | Sets | Units / range |
|---|---|---|
| a | Target angle | −180 to 180° (0 = horizontal) |
| p | Kp | duty counts per degree |
| i | Ki | duty counts per degree per ms |
| d | Kd | duty counts per °/s |
| m | Integrator cap | 0–4500 duty counts |
| c | Closed loop on/off | 1 / 0 (0 also turns the motor off) |
| n | Manual duty cycle (turns closed loop off) | 0–4500 |
| t | Plot timestep | ISR samples per plotted point |

Table 3: Serial commands

### Demo Sequence

Pressing and releasing a button on GPIO 2 (wired to ground, internal pull-up) runs the four-angle test with the current gains, with no reset needed. The ISR debounces the button with a counter that must hold for 20 ms, and times the sequence in 1 ms ISR ticks:

| Time | Target and controller |
|---|---|
| Button held | Closed loop off, duty 0, integrator cleared; arm hangs at −90° |
| Release (t = 0) | Target 0°, closed loop on |
| t = 5 s | Target +30° |
| t = 10 s | Target −30° |
| t = 15 s | Target 0°; sequence ends and the controller keeps holding horizontal |

The VGA readout shows the demo status and elapsed time, so each screen dump shows where in the sequence it was taken.

### Final Wiring

| Signal | RP2350 pin | Notes |
|---|---|---|
| VGA Hsync / Vsync | GPIO 16 / 17 | PIO state machines |
| VGA Green lo / Green hi / Blue / Red | GPIO 18 / 19 / 20 / 21 | 470 Ω / 330 Ω / 330 Ω / 330 Ω |
| MPU-6050 SDA / SCL | GPIO 8 / 9 | I2C0, 400 kHz |

| Signal | RP2350 pin | Notes |
|---|---|---|
| Motor PWM | GPIO 4 | slice 2 channel A, to optoisolator |
| ISR timing | GPIO 14 | toggles every ISR |
| Frame timing | GPIO 15 | toggles every VGA frame |
| Demo button | GPIO 2 | pushbutton to GND, internal pull-up, 20 ms debounce |

Table 4: RP2350 wiring connections

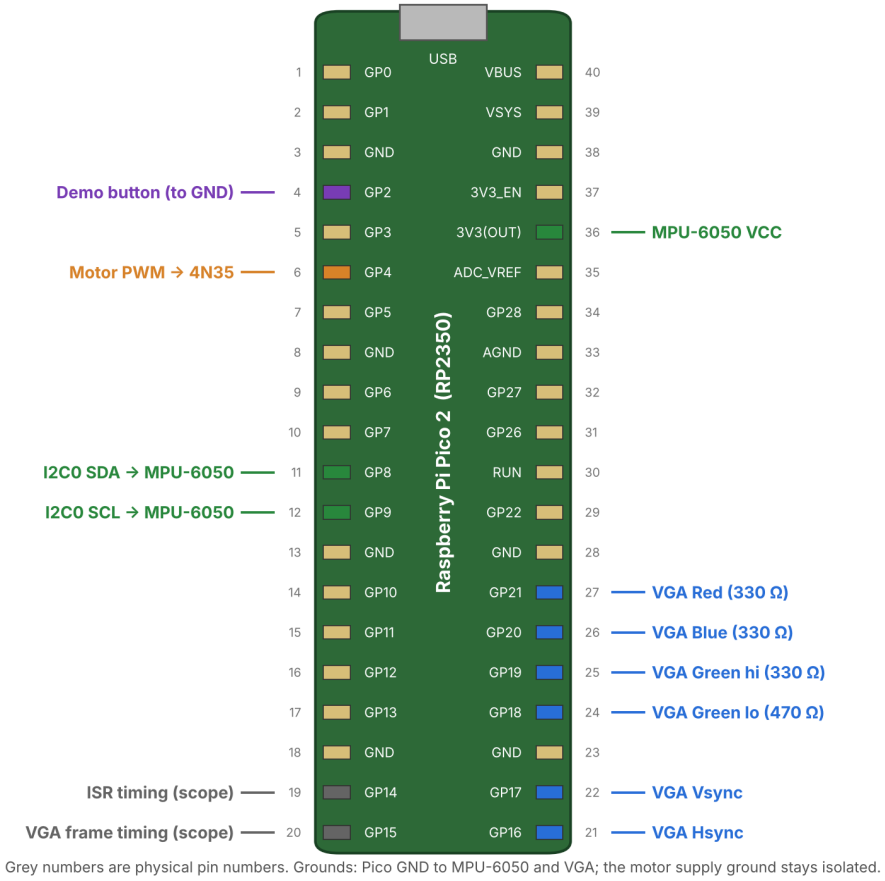



*Figure 13: Pico 2 pinout with our pins, colored by subsystem.*

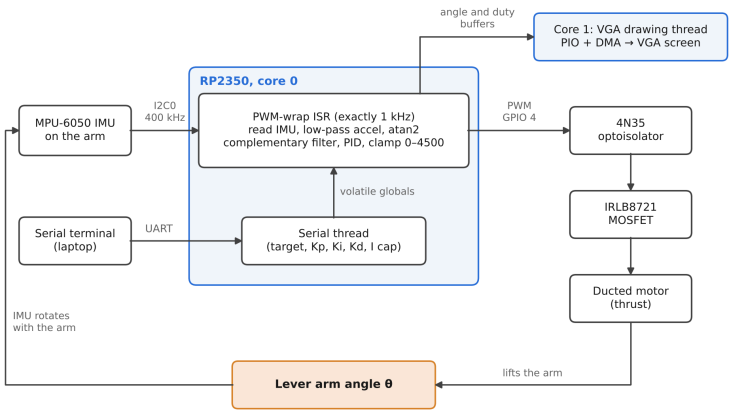



*Figure 14: System block diagram. Core 0 closes the loop at 1 kHz and runs the serial thread; core 1 only draws.*

[Figure 15: TODO close-up photo of the breadboard: Pico 2, motor driver PCB, VGA board and wiring]

## Documentation

Where to find the drawings and listings the course policy asks for:

| Item | Where |
|---|---|
| Motor circuit schematic | Figure 9 |
| System block diagram | Figure 14 |
| Complementary filter | Figures 2 and 3 |
| VGA adapter PCB and driver | Figures 10 and 11 |
| Pinout and pin assignments | Figure 13, Table 4 |
| Rig photos | Figures 1 and 15 |
| IMU register settings | Table 2 |
| Serial commands | Table 3 |
| VGA screen dumps | Figures 12, 16 and 17 |
| Commented code listing | Appendix A |
| AI prompt log | Appendix B |

## Results

Our final gains were Kp = [TODO], Ki = [TODO], Kd = [TODO] with an integrator cap of [TODO] duty counts. One set of gains held the arm stable from [TODO]° to [TODO]°.

### Testing Each PID Term

Answer 3 (Answers to Specific Lab Questions) explains how we chose the starting gains and the tuning order. Before tuning, we tested each term on its own.

**Unit tests for each term.** Before tuning, we checked each term in isolation by hand:

- **P:** pull the arm away from the target; the motor speeds up as the error grows. [TODO: observed?]
- **I:** with Kp = 0, hold the arm about 5° off target; the motor ramps up steadily until it reaches the cap. [TODO]
- **D:** hold the arm up and drop it; the motor kicks on and slows the fall. [TODO]
- **Windup:** with the cap removed, holding the arm down then releasing it caused [TODO]; with the cap, [TODO].

### Stable Angle Range

With the final gains, the arm was stable from [TODO: min]° to [TODO: max]°. Answer 4 explains how we measured this range and why it is limited.

### Settling Times

| Step (from → to) | Settling time (s) | Overshoot (°) |
|---|---|---|
| −90° → 0° | [TODO] | [TODO] |
| 0° → +30° | [TODO] | [TODO] |
| +30° → −30° | [TODO] | [TODO] |
| −30° → 0° | [TODO] | [TODO] |
| [TODO: max angle step] | [TODO] | [TODO] |

Table 5: Settling times with final gains (settled = inside ±2° of the target and staying there)

[Figure 16: TODO VGA screen dump of the four-angle demo sequence: angle and target (top), low-passed motor command (bottom)]

[Figure 17: TODO VGA screen dump of a step at a high angle]

## Conclusion

[TODO: one-line result, e.g. "Our helicopter hit all four demo angles with one set of gains and settled within ___ s."] We learned how to fuse two imperfect sensors with a complementary filter, how each PID term shapes the response (and how the analytical gain and phase margin picture explains what we saw while tuning), and why a motor circuit needs optical isolation, a flyback diode and a bypass capacitor. Most of our debugging time went to hardware and noise rather than code: [TODO: e.g. the stuck I2C bus, noisy angle traces, grounding]. [TODO: what you would do differently.]

## Answers to Specific Lab Questions

Bracketed blanks come from the final demo run.

## 1. Schematic of the circuit we built

Figure 9. GPIO 4 drives the 4N35's LED through 330 Ω. Its phototransistor switches the 12 V gate supply (through a kill switch) onto the IRLB8721's gate, with a 10 kΩ gate pull-down and a 1 MΩ base resistor. The motor runs from a separate 5 V supply with a 1N4001 flyback diode and a 0.1 µF ceramic capacitor across it. The two sides share no ground.

## 2. VGA screen dumps of motor control and actual angle

Figure 16 shows the four-angle demo and Figure 17 a high-angle step. In each, the top trace is the angle (yellow) against the target (blue) and the bottom trace is the low-passed motor command (orange). While the button is held, the command is 0 and the arm hangs at −90°. The arm model predicts that +30° and −30° need about the same thrust, since gravity's torque scales with cos 30° = 0.87 at both, and less than horizontal. [TODO: compare with Figure 16.]

## 3. How we selected the three PID gains

We scaled each starting value so its term gives a sensible duty cycle:

- **Kp ≈ 4500 / 90 ≈ 50:** a 90° error (hanging down, target horizontal) should call for near-full duty.
- **Kd in the same decade as Kp:** the gyro rate in °/s is the same order as the angle error in degrees. The notes suggest thousands, but that assumes differencing noisy ADC samples.
- **Ki ≈ 1/32:** the integrator adds Ki × error 1000 times per second.

We then followed the notes' procedure:

1. With P only, raise Kp until the arm oscillates steadily. Figure 12 shows this at Kp = 120, with a period of about 0.8 s.
2. Add Kd to damp the oscillation. Alternate raising Kp and Kd until the response is fast with little overshoot.
3. Add a small Ki to remove steady-state error, with i_max just above the duty that holds the arm horizontal.

[TODO: what we saw at each step.] Final gains: Kp = [TODO], Ki = [TODO], Kd = [TODO], i_max = [TODO].

## 4. Maximum and minimum angles with stable behavior

The arm was stable from [TODO: min]° to [TODO: max]° (0° = horizontal). We found the limits by stepping the target in 10° increments each way until the arm no longer settled. The plant model explains why the range is limited:

- **Below horizontal**, gravity's stiffness is positive, so the arm acts like a stable pendulum. Near −90° the thrust needed approaches zero and the motor barely spins.
- **Above horizontal**, the stiffness is negative, so the controller has to supply all of the stability.
- **The plant gain** is proportional to $2K_T \cdot \omega_0$. Away from horizontal the motor spins slower, so each duty count moves the arm less, and one set of gains can't be ideal everywhere.
- **Output limits:** the motor only pushes the arm up, and the command is capped at 4500.

Past the maximum we saw [TODO]; below the minimum, [TODO].

### 5. Settling times at different angles

Settling means entering and staying within ±2° of the target. We measured it from VGA dumps as pixels × timestep (10 ms, or 40 ms with t = 40), checked against the demo-time readout. Table 5 lists each step. In summary, [TODO].

We expect steps that end above horizontal to settle more slowly, since gravity no longer helps. Downward steps should be asymmetric: the motor can only cut thrust, so gravity drives the descent. [TODO: confirm against Table 5.]

### 6. Heavily commented code listing

Appendix A lists imu_demo.c, mpu6050.c, mpu6050.h and CMakeLists.txt. Every function has a header comment, and changes from the course demo are marked ADDED, CHANGED or FIXED.

### 7. AI prompt log

Appendix B (Table 6) lists each AI session with the words exchanged and the code changes suggested and accepted.

## Work Distribution

Amber, Ivan, and Zarif worked together on all parts of the lab. We frequently met outside of our lab sections, and most of our work was done in person in the lab. Outside of the lab, we communicated frequently to ensure that we were all aware of the progress that had been made and what the immediate next steps were. The contents of this report were written by us.

## Appendix A: Heavily Commented Code

The project builds from four of our files plus the course's VGA driver and protothreads header, which we used unchanged. Every change from the course's MPU-6050 demo is marked ADDED, CHANGED or FIXED in the comments.

### A.1 imu_demo.c (main program)

```
/**
 * ECE 5730 Lab 3: PID control of a 1D helicopter
 * Amber Li, Ivan Huang, Zarif Karim
 * Based on the MPU6050 IMU demo by V. Hunter Adams (vha3@cornell.edu)
 *
 * PROGRAM OVERVIEW
 *  - Core 0, PWM-wrap ISR (exactly 1 kHz): reads the MPU6050 over I2C,
 *    estimates the arm angle with a complementary filter, runs the PID
 *    controller, sets the motor PWM duty cycle, runs the demo-button
 *    sequence and fills the display buffers.
 *  - Core 0, serial protothread: reads one-letter commands that set the
 *    target angle, gains, integrator cap, mode and plot speed.
 *  - Core 1, draw protothread: redraws the VGA plots and text every frame.
 * All ISR math is 15.16 fixed point (fix15, see mpu6050.h) except atan2.
 * Angle convention: 0 deg = arm horizontal, -90 deg = hanging down,
 * positive = above horizontal.
 *
 * ORIGINAL DEMO DESCRIPTION
 * This demonstration utilizes the MPU6050.
```

```
* It gathers raw accelerometer/gyro measurements, scales
* them, and plots them to the VGA display. The top plot
* shows the target and actual (complementary) angle, bottom
* plot shows the fan duty cycle (PID output).
* CHANGED: top plot was gyro, bottom plot was accelerometer.
*
* HARDWARE CONNECTIONS
*  - GPIO 16 ---> VGA Hsync
*  - GPIO 17 ---> VGA Vsync
*  - GPIO 18 ---> 470 ohm resistor ---> VGA Green
*  - GPIO 19 ---> 330 ohm resistor ---> VGA Green
*  - GPIO 20 ---> 330 ohm resistor ---> VGA Blue
*  - GPIO 21 ---> 330 ohm resistor ---> VGA Red
*  - RP2040 GND ---> VGA GND
*  - GPIO 8 ---> MPU6050 SDA
*  - GPIO 9 ---> MPU6050 SCL
*  - 3.3v ---> MPU6050 VCC
*  - RP2040 GND ---> MPU6050 GND
*  - GPIO 4 ---> PWM_OUT
*  - GPIO 2 ---> demo pushbutton ---> GND (internal pull-up, pressed = low)
*  - GPIO 14 --> toggles every ISR (scope timing)
*  - GPIO 15 --> toggles every VGA frame (scope timing)
*/


// Include standard libraries
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <string.h>
// Include PICO libraries
#include "pico/stdlib.h"
#include "pico/multicore.h"
// Include hardware libraries
#include "hardware/pwm.h"
#include "hardware/dma.h"
#include "hardware/irq.h"
#include "hardware/adc.h"
#include "hardware/pio.h"
#include "hardware/i2c.h"
#include "hardware/clocks.h"
#include "hardware/sync.h"
// Include custom libraries
#include "VGA/vga16_graphics_v3.h"
#include "mpu6050.h"
#include "pt_cornell_rp2040_v1_4.h"

// We will start drawing at column 81
#define LEFT_EDGE 81

// Rescale the measurements for display
#define OLD_RANGE 500.  // (+/- 250)
#define NEW_RANGE 150.  // (looks nice on VGA)
#define OLD_MIN -250.
#define OLD_MAX 250.
#define ACCEL_SCALE 120.0
```

```
// Angle plot range (degrees)
#define ANGLE_MIN 0
#define ANGLE_RANGE 180
#define RAD2DEG 57.2957795f

// Place the plots
#define TOP_PLOT_BOTTOM 230
#define BOTTOM_PLOT_BOTTOM 430
#define PLOT_WIDTH 530

// Arrays in which raw measurements will be stored
fix15 acceleration[3], gyro[3];

// Arrays for circular display buffer
fix15 ax[PLOT_WIDTH] ;
fix15 ay[PLOT_WIDTH] ;
fix15 az[PLOT_WIDTH] ;
fix15 gx[PLOT_WIDTH] ;
fix15 gy[PLOT_WIDTH] ;
fix15 gz[PLOT_WIDTH] ;

// ===== ADDED: complementary filter =====

// Low-passed accelerometer measurements (15.16 fixed point, g's)
fix15 filtered_ax, filtered_az ;

// Angle estimates (15.16 fixed point, degrees)
fix15 accel_angle ;          // angle from accelerometer only (noisy, no drift)
fix15 gyro_angle_delta ;     // angle change from gyro this timestep
fix15 complementary_angle ;  // fused angle estimate

// Circular display buffers for the angle estimates
fix15 accel_angle_buf[PLOT_WIDTH] ;
fix15 comp_angle_buf[PLOT_WIDTH] ;
// ===== END ADDED =====

// ===== ADDED: circular display buffer for the fan duty cycle (PID output) =====
int duty_buf[PLOT_WIDTH] ;
// ===== END ADDED =====

// ===== ADDED: low-passed motor command for display =====
// The handout requires the PLOTTED motor command to be low-passed. First-order
// IIR with a time constant of 16 PWM periods (16 ms at 1 kHz), from the Display page:
//   motor_disp = motor_disp + ((control - motor_disp) >> 4)
// Only the plot uses this; the motor itself still gets the raw PID output.
volatile int motor_disp = 0 ;
// ===== END ADDED =====

// Pointer to head of buffer
volatile int head = 0 ;

// character array
char screentext[80];   // CHANGED: was 40, controller readout line is longer

// draw speed
```

```
volatile int threshold = 10 ;

// Some macros for max/min/abs
#define min(a,b) ((a<b) ? a:b)
#define max(a,b) ((a<b) ? b:a)
#define abs(a) ((a>0) ? a:-a)

// semaphore
static struct pt_sem vga_semaphore ;

// Some paramters for PWM
#define WRAPVAL 6000
#define CLKDIV  25.0
// ADDED: max allowed fan duty cycle (closed loop and manual 'n'), below WRAPVAL
#define DUTY_MAX 4500
// GPIO we're using for PWM
#define PWM_OUT 4
uint slice_num ;
// PWM duty cycle
volatile int control ;
volatile int old_control ;

// ===== ADDED: proportional controller =====
volatile int closed_loop = 0 ;                // 0 = manual duty ('n'), 1 = PID control
volatile fix15 desired_angle = 0 ;            // target angle (degrees, 15.16)
volatile fix15 Kp = int2fix15(100) ;          // gain (duty counts per degree, 15.16)
fix15 angle_error ;                           // desired - measured (degrees, 15.16)
// ===== END ADDED =====

// ===== ADDED: derivative control =====
volatile fix15 Kd = int2fix15(0) ;            // gain (duty counts per deg/s, 15.16), 0 = off
// ===== END ADDED =====

// ===== ADDED: integral control =====
volatile fix15 Ki = int2fix15(0) ;            // gain (duty counts per deg, per 1 ms sample, 15.16), 0 = off
volatile int i_max = 3000 ;                   // anti-windup cap on the I term (duty counts)
volatile fix15 i_term = 0 ;                   // accumulated Ki*error (duty counts, 15.16)
// ===== END ADDED =====

// ===== ADDED: button-triggered demo sequence =====
// While the button is held: motor off, arm hangs straight down.
// On release (t = 0): target 0 deg, closed loop on.
// t = 5 s: +30 deg.  t = 10 s: -30 deg.  t = 15 s: back to 0 deg, sequence ends
// (controller stays on, holding horizontal).
// Runs inside the 1 kHz ISR, so one ISR call = exactly 1 ms of demo time.
#define BUTTON_PIN    2
#define DEBOUNCE_MS   20       // button must read the same for 20 ms to count
#define DEMO_IDLE     0
#define DEMO_HELD     1        // button is down, motor off
#define DEMO_RUNNING  2        // button released, stepping through targets
volatile int demo_state = DEMO_IDLE ;
volatile int demo_ms = 0 ;     // ms since the button was released
static int button_count = 0 ;  // debounce integrator, 0..DEBOUNCE_MS
static int button_down = 0 ;   // debounced button state (1 = pressed)
// ===== END ADDED =====
```

```
// ----------------------------------------------------------------------
// drawAxes(): draws the static parts of both plots (axis lines, tick
// marks and their labels). Called once per frame by the draw thread,
// right after the plot area is cleared.
// Top plot: angle, -250 to +250 deg. Bottom plot: duty, 0 to DUTY_MAX.
// ----------------------------------------------------------------------
void drawAxes() {
   // Draw the static aspects of the display
   setTextSize(1) ;
   setTextColor(WHITE);

   // Draw bottom plot
   drawHLine(75, 430, 5, CYAN) ;
   drawHLine(75, 355, 5, CYAN) ;
   drawHLine(75, 280, 5, CYAN) ;
   drawVLine(80, 280, 150, CYAN) ;
   // CHANGED: labels are now duty cycle (0 to DUTY_MAX), were accel (-2 to +2 g)
   sprintf(screentext, "%d", DUTY_MAX/2) ;
   setCursor(45, 350) ;
   writeString(screentext) ;
   sprintf(screentext, "%d", DUTY_MAX) ;
   setCursor(45, 280) ;
   writeString(screentext) ;
   sprintf(screentext, "0") ;
   setCursor(50, 425) ;
   writeString(screentext) ;
   sprintf(screentext, "Duty") ;
   setCursor(45, 300) ;
   writeString(screentext) ;

   // Draw top plot
   drawHLine(75, 230, 5, CYAN) ;
   drawHLine(75, 155, 5, CYAN) ;
   drawHLine(75, 80, 5, CYAN) ;
   drawVLine(80, 80, 150, CYAN) ;
   sprintf(screentext, "0") ;
   setCursor(50, 150) ;
   writeString(screentext) ;
   sprintf(screentext, "+250") ;
   setCursor(45, 75) ;
   writeString(screentext) ;
   sprintf(screentext, "-250") ;
   setCursor(45, 225) ;
   writeString(screentext) ;
}

volatile int throttle = 10 ;

// ----------------------------------------------------------------------
// on_pwm_wrap(): the control loop. The PWM slice raises this interrupt
// each time its counter wraps (every 6001 counts at 150 MHz / 25 = 1 kHz),
// so the controller runs exactly once per PWM period with no separate
// timer. Order of work each call:
//   1. clear the IRQ flag, toggle GPIO 14 for scope timing
//   2. read raw accel + gyro over I2C (about 0.4 ms of the 1 ms budget)
```

```
//   3. low-pass accel, compute accel angle with atan2
//   4. complementary filter: 0.999*(angle + gyro*dt) + 0.001*accel angle
//   5. demo button debounce + timed target sequence
//   6. PID (if closed loop), clamp to 0..DUTY_MAX, write the PWM level
//   7. low-pass the command for display, fill the circular plot buffers
// Runs on core 0 (the IRQ was enabled from core 0 in main()).
// ------------------------------------------------------------------------
void on_pwm_wrap() {

    // Clear the interrupt flag that brought us here
    pwm_clear_irq(pwm_gpio_to_slice_num(5));
    gpio_put(14, !gpio_get(14)) ;

    // Read the IMU
    // NOTE! This is in 15.16 fixed point. Accel in g's, gyro in deg/s
    // If you want these values in floating point, call fix2float15() on
    // the raw measurements.
    mpu6050_read_raw(acceleration, gyro);

    // ===== ADDED: low pass filter on accelerometer (first-order IIR, alpha = 1/16) =====
    filtered_ax = filtered_ax + ((acceleration[0] - filtered_ax) >> 4);
    filtered_az = filtered_az + ((acceleration[2] - filtered_az) >> 4);
    // ===== END ADDED =====

    // Accelerometer angle (degrees - 15.16 fixed point)
    // NO SMALL ANGLE APPROXIMATION --> ADDED LOW PASS FILTER
    // IMU rotates about +y axis
    // FIXED: removed +PI so flat (arm horizontal) = 0 deg, range -180 to +180
    accel_angle = multfix15(float2fix15(atan2(-filtered_ax, filtered_az)), oneeightyoverpi);

    // Gyro angle delta (measurement times timestep) (15.16 fixed point)
    // gyro y for rotation about +y axis
    gyro_angle_delta = multfix15(gyro[1], zeropt001) ;

    // Complementary angle (degrees - 15.16 fixed point)
    // gyro and accel share the same sign convention so + instead between (comp ang + gyro ang)
    complementary_angle = multfix15(complementary_angle + gyro_angle_delta, zeropt999) +
multfix15(accel_angle, zeropt001);

    // ===== ADDED: demo button (debounce + timed target sequence) =====
    // Debounce by integration: count up while the pin reads pressed (low),
    // down while it reads released. The state only flips at the ends of the
    // range, so contact bounce shorter than DEBOUNCE_MS is ignored.
    if (!gpio_get(BUTTON_PIN)) { if (button_count < DEBOUNCE_MS) button_count++ ; }
    else                       { if (button_count > 0)          button_count-- ; }
    if (button_count == DEBOUNCE_MS) button_down = 1 ;
    else if (button_count == 0)      button_down = 0 ;

    if (button_down) {
        // Held: motor off so the arm hangs straight down (-90 deg).
        // closed_loop = 0 also clears the integrator below.
        demo_state = DEMO_HELD ;
        closed_loop = 0 ;
        control = 0 ;
    }
    else if (demo_state == DEMO_HELD) {
```

```
    // Just released: t = 0, target horizontal, controller on
    demo_state = DEMO_RUNNING ;
    demo_ms = 0 ;
    desired_angle = int2fix15(0) ;
    closed_loop = 1 ;
}
else if (demo_state == DEMO_RUNNING) {
    demo_ms++ ;
    if      (demo_ms ==  5000) desired_angle = int2fix15(30) ;   // t = 5 s
    else if (demo_ms == 10000) desired_angle = int2fix15(-30) ;  // t = 10 s
    else if (demo_ms == 15000) {                                 // t = 15 s
        desired_angle = int2fix15(0) ;
        demo_state = DEMO_IDLE ;   // done; closed loop keeps holding 0 deg
    }
}
// ===== END ADDED =====

// ===== ADDED: proportional control =====
// ===== CHANGED: now PID control (derivative and integral terms added below) =====
if (closed_loop) {
    // Positive error = arm is below the target, so more thrust is needed
    angle_error = desired_angle - complementary_angle ;
    // Kp * error in 64 bits (avoids overflow), >>32 converts 15.16 * 15.16 to an integer duty cycle
    long long p_term = ((long long)Kp) * ((long long)angle_error) ;
    // ===== ADDED: derivative term =====
    // d(error)/dt = -d(angle)/dt = -gyro[1] (gyro y is the arm's angular rate in deg/s).
    // Using the gyro directly avoids the noise of differencing the angle estimate.
    // Opposes motion: arm rising fast -> less thrust, arm falling fast -> more thrust.
    long long d_term = ((long long)Kd) * ((long long)gyro[1]) ;
    // ===== END ADDED =====
    // ===== ADDED: integral term =====
    // Accumulate Ki*error every sample (rather than summing error and scaling later),
    // so the sum is already in duty counts and can't overflow 15.16, and changing Ki
    // doesn't make the output jump. At steady state P and D are ~0, so the I term
    // alone supplies the duty that holds the arm up against gravity.
    i_term += multfix15(Ki, angle_error) ;
    // Anti-windup: cap at i_max (set it a bit above the duty that holds the arm at the
    // target). Floor at 0 because the fan can't pull down, so negative I only adds lag.
    if (i_term > int2fix15(i_max)) i_term = int2fix15(i_max) ;
    if (i_term < 0) i_term = 0 ;
    // ===== END ADDED =====
    int u = (int)((p_term - d_term) >> 32) + fix2int15(i_term) ;   // CHANGED: was Kp * error only, then P - D
    // The fan can only push one way, so clamp to the valid duty range
    if (u < 0) u = 0 ;
    if (u > DUTY_MAX) u = DUTY_MAX ;   // CHANGED: was WRAPVAL (6000)
    control = u ;
}
// ===== ADDED: clear the integrator whenever the controller is off =====
// so it starts from 0 the next time closed loop is turned on
else {
    i_term = 0 ;
}
// ===== END ADDED =====
// ===== END ADDED =====

// Update fan duty cycle
```

```
    if (control!=old_control) {
        old_control = control ;
        pwm_set_chan_level(slice_num, PWM_CHAN_A, control);
    }

    // ===== ADDED: low-pass the motor command for display (every ms, not just
    // when a sample is plotted, so the filter sees every PID output) =====
    motor_disp = motor_disp + ((control - motor_disp) >> 4) ;
    // ===== END ADDED =====

    // Increment drawspeed controller
    throttle += 1 ;
    // Maintain a circular buffer for display
    if ((throttle >= threshold)) {
        // Zero drawspeed controller
        throttle = 0 ;

        // Replace oldest data in accel arrays with new sample
        ax[head] = acceleration[0] ;
        ay[head] = acceleration[1] ;
        az[head] = acceleration[2] ;

        // Replace oldest data in gyro arrays with new sample
        gx[head] = gyro[0] ;
        gy[head] = gyro[1] ;
        gz[head] = gyro[2] ;

        // ===== ADDED: store complementary filter angle estimates for display =====
        accel_angle_buf[head] = accel_angle ;
        comp_angle_buf[head] = complementary_angle ;
        // ===== END ADDED =====

        // ===== ADDED: store fan duty cycle for display =====
        // CHANGED: store the low-passed command (was raw control)
        duty_buf[head] = motor_disp ;
        // ===== END ADDED =====

        // Move the pointer to the oldest data
        head = ((head+1) == PLOT_WIDTH) ? 0 : (head + 1) ;

    }
}

// ---------------------------------------------------------------------
// protothread_draw (core 1): redraws the VGA screen once per frame.
// Waits for the VGA driver's buffer-swap signal, clears the plot area,
// redraws the axes, then plots the circular buffers oldest -> newest:
//   top:    complementary angle (yellow) + target line (light blue)
//   bottom: low-passed motor command (orange)
// and finally the numeric readout (angles, gains, duty, mode, demo time).
// Running on core 1 means drawing can never delay the control ISR.
// ---------------------------------------------------------------------
static PT_THREAD (protothread_draw(struct pt *pt))
{
    PT_BEGIN(pt) ;
```

```
  static int start ;
  static int i ;

  // Draw axes
  drawAxes() ;
  // Copy to other buffer
  copy_buffer_to_other() ;

  while(1) {
    // Wait for a buffer swap
    PT_YIELD_UNTIL(pt, draw_start_signal()) ;
    gpio_put(15, !gpio_get(15)) ;

    // Clear the buffer, draw the axes
    clearLowFrame(0, BLACK) ;
    drawAxes() ;

    // Plot data oldest-->newest
    start = head ;
    for (i = 0; i < PLOT_WIDTH; i++) {
      // ===== CHANGED: bottom plot is now duty cycle (was ax/ay/az) =====
      // 0 duty at the bottom of the plot, DUTY_MAX at the top
      drawPixel(LEFT_EDGE + i, (BOTTOM_PLOT_BOTTOM -
        (int)(NEW_RANGE*((float)duty_buf[start]/DUTY_MAX))), ORANGE) ;
      // ===== END CHANGED =====

      // ===== CHANGED: top plot shows only the actual (complementary) angle =====
      // (gyro x/y/z and accel-only angle traces removed; target line drawn below)
      // Angles rotate about +y axis within the range ~[-90, 90] so 0 degree is when the IMU is parallel to the floor
      drawPixel(LEFT_EDGE + i, (TOP_PLOT_BOTTOM -
        (int)(NEW_RANGE*((float)((fix2float15(comp_angle_buf[start]))-OLD_MIN)/OLD_RANGE))),
YELLOW) ;
      // ===== END CHANGED =====

      start = ((start+1)==PLOT_WIDTH) ? 0 : (start+1) ;
    }

    // ===== ADDED: target angle line on top plot =====
    drawHLine(LEFT_EDGE, (TOP_PLOT_BOTTOM -
      (int)(NEW_RANGE*((float)((fix2float15(desired_angle))-OLD_MIN)/OLD_RANGE))), PLOT_WIDTH,
LIGHT_BLUE) ;
    // ===== END ADDED =====

    // ===== ADDED: numeric readout of angle estimates =====
    setTextSize(2) ;
    setTextColor(MAGENTA) ;
    setCursor(85, 20) ;
    sprintf(screentext, "Accel Ang: %6.1f", fix2float15(accel_angle)) ;
    writeString(screentext) ;
    setTextColor(YELLOW) ;
    setCursor(330, 20) ;
    sprintf(screentext, "Comp Ang: %6.1f", fix2float15(complementary_angle)) ;
    writeString(screentext) ;
    setTextSize(1) ;
    // ===== END ADDED =====
```

```
      // ===== ADDED: controller readout =====
      setTextColor(LIGHT_BLUE) ;
      setCursor(85, 45) ;
      // CHANGED: added Kd to the readout
      sprintf(screentext, "Target: %6.1f  Kp: %6.1f  Kd: %6.1f  Duty: %4d  %s",
         fix2float15(desired_angle), fix2float15(Kp), fix2float15(Kd), control, closed_loop ? "CLOSED LOOP" :
"MANUAL") ;
      writeString(screentext) ;
      // ===== ADDED: integral readout (second line) =====
      setCursor(85, 57) ;
      sprintf(screentext, "Ki: %7.4f  I max: %4d  I term: %6.1f",
         fix2float15(Ki), i_max, fix2float15(i_term)) ;
      writeString(screentext) ;
      // ===== ADDED: demo status, so screen dumps show where in the sequence we are =====
      setCursor(330, 57) ;
      if (demo_state == DEMO_HELD)
         sprintf(screentext, "DEMO: button held, motor off   ") ;
      else if (demo_state == DEMO_RUNNING)
         sprintf(screentext, "DEMO: t = %5.1f s              ", demo_ms / 1000.0f) ;
      else
         sprintf(screentext, "                               ") ;
      writeString(screentext) ;
      // ===== END ADDED =====
      // ===== END ADDED =====
      // ===== END ADDED =====
   }

   PT_END(pt) ;
}

// ---------------------------------------------------------------------
// protothread_serial (core 0): the user interface. Waits (without
// blocking the ISR) for a one-letter command, then prompts for a value:
//   t = plot timestep (ISR samples per plotted point)
//   n = manual duty cycle (turns closed loop off)
//   a = target angle (deg)   p = Kp   d = Kd   i = Ki
//   m = integrator cap (duty counts)   c = closed loop on (1) / off (0)
// Values are range-checked, then written straight into the volatile
// globals the ISR reads, so they take effect on the next 1 ms cycle.
// ---------------------------------------------------------------------
static PT_THREAD (protothread_serial(struct pt *pt))
{
   PT_BEGIN(pt) ;
   static char classifier ;
   static int test_in ;
   static float float_in ;
   while(1) {
      sprintf(pt_serial_out_buffer, "input a command: ");
      serial_write ;
      // spawn a thread to do the non-blocking serial read
      serial_read ;
      // convert input string to number
      sscanf(pt_serial_in_buffer,"%c", &classifier) ;

      // num_independents = test_in ;
      if (classifier=='t') {
```

```
    sprintf(pt_serial_out_buffer, "timestep: ");
    serial_write ;
    serial_read ;
    // convert input string to number
    sscanf(pt_serial_in_buffer,"%d", &test_in) ;
    if (test_in > 0) {
      threshold = test_in ;
    }
  }
  // ===== ADDED: 'n' command sets fan duty cycle (from PWM_Demo) =====
  else if (classifier=='n') {
    // CHANGED: limit is DUTY_MAX (was WRAPVAL)
    sprintf(pt_serial_out_buffer, "duty cycle (0-%d): ", DUTY_MAX);
    serial_write ;
    serial_read ;
    // convert input string to number
    sscanf(pt_serial_in_buffer,"%d", &test_in) ;
    if (test_in >= 0 && test_in <= DUTY_MAX) {
      closed_loop = 0 ;   // ADDED: manual duty turns off the controller
      control = test_in ;
    }
  }
  // ===== END ADDED =====
  // ===== ADDED: controller commands =====
  // 'a' = target angle, 'p' = Kp, 'd' = Kd, 'i' = Ki, 'm' = I term cap, 'c' = closed loop on (1) / off (0)
  else if (classifier=='a') {
    sprintf(pt_serial_out_buffer, "target angle (-180 to 180 deg): ");
    serial_write ;
    serial_read ;
    sscanf(pt_serial_in_buffer,"%f", &float_in) ;
    if (float_in >= -180.0f && float_in <= 180.0f) {
      desired_angle = float2fix15(float_in) ;
    }
  }
  else if (classifier=='p') {
    sprintf(pt_serial_out_buffer, "Kp (duty counts per degree): ");
    serial_write ;
    serial_read ;
    sscanf(pt_serial_in_buffer,"%f", &float_in) ;
    if (float_in >= 0.0f && float_in <= 6000.0f) {
      Kp = float2fix15(float_in) ;
    }
  }
  // ===== ADDED: 'd' command sets Kd =====
  else if (classifier=='d') {
    sprintf(pt_serial_out_buffer, "Kd (duty counts per deg/s): ");
    serial_write ;
    serial_read ;
    sscanf(pt_serial_in_buffer,"%f", &float_in) ;
    if (float_in >= 0.0f && float_in <= 6000.0f) {
      Kd = float2fix15(float_in) ;
    }
  }
  // ===== END ADDED =====
  // ===== ADDED: 'i' sets Ki, 'm' sets the I term cap =====
  else if (classifier=='i') {
```

```
            sprintf(pt_serial_out_buffer, "Ki (duty counts per deg per ms, e.g. 0.03): ");
            serial_write ;
            serial_read ;
            sscanf(pt_serial_in_buffer,"%f", &float_in) ;
            if (float_in >= 0.0f && float_in <= 100.0f) {
               Ki = float2fix15(float_in) ;
            }
         }
         else if (classifier=='m') {
            sprintf(pt_serial_out_buffer, "I term max (0-%d): ", DUTY_MAX);
            serial_write ;
            serial_read ;
            sscanf(pt_serial_in_buffer,"%d", &test_in) ;
            if (test_in >= 0 && test_in <= DUTY_MAX) {
               i_max = test_in ;
            }
         }
         // ===== END ADDED =====
         else if (classifier=='c') {
            sprintf(pt_serial_out_buffer, "closed loop (1 = on, 0 = off): ");
            serial_write ;
            serial_read ;
            sscanf(pt_serial_in_buffer,"%d", &test_in) ;
            if (test_in == 0) {
               closed_loop = 0 ;
               control = 0 ;   // fan off when leaving closed loop
            }
            else if (test_in == 1) {
               closed_loop = 1 ;
            }
         }
         // ===== END ADDED =====
      }
      PT_END(pt) ;
}

// Entry point for core 1: set up the frame-timing pin (GPIO 15) and run
// the draw thread on core 1's own protothread scheduler.
void core1_entry() {
   gpio_init(15) ;
   gpio_set_dir(15, GPIO_OUT) ;
   gpio_put(15, 0) ;
   pt_add_thread(protothread_draw) ;
   pt_schedule_start ;
}

// ------------------------------------------------------------------------
// main(): one-time setup on core 0, in dependency order:
//   clock -> stdio -> VGA -> I2C bus recovery -> I2C + MPU6050 init ->
//   seed filters from the first reading -> demo button -> PWM + its IRQ
//   (this starts the 1 kHz control loop) -> launch core 1 (drawing) ->
//   run the serial thread on core 0 forever.
// ------------------------------------------------------------------------
int main() {

   // Set the system clock to 150 MHz (the RP2350 default; the PWM
```

```
// divider and wrap value above assume this frequency)
set_sys_clock_khz(150000, true) ;

// Initialize stdio
stdio_init_all();

// Initialize VGA
initVGA() ;

// ===== ADDED: I2C bus recovery =====
// If the Pico resets mid-transfer, the MPU6050 (which stays powered) can be
// left holding SDA low, and mpu6050_reset() then blocks forever. Clocking SCL
// up to 9 times lets the IMU finish its byte and release SDA.
gpio_init(SDA_PIN) ;
gpio_init(SCL_PIN) ;
gpio_set_dir(SDA_PIN, GPIO_IN) ;
gpio_pull_up(SDA_PIN) ;
gpio_set_dir(SCL_PIN, GPIO_OUT) ;
gpio_put(SCL_PIN, 1) ;
sleep_us(10) ;
for (int k = 0; k < 9 && !gpio_get(SDA_PIN); k++) {
    gpio_put(SCL_PIN, 0) ;
    sleep_us(5) ;
    gpio_put(SCL_PIN, 1) ;
    sleep_us(5) ;
}
printf("\n\rI2C bus %s\n\r", gpio_get(SDA_PIN) ? "free" : "STUCK (SDA held low)") ;
// ===== END ADDED =====

////////////////////////////////////////////////////////////////////////
////////////////////////// I2C CONFIGURATION ///////////////////////////
i2c_init(I2C_CHAN, I2C_BAUD_RATE) ;
gpio_set_function(SDA_PIN, GPIO_FUNC_I2C) ;
gpio_set_function(SCL_PIN, GPIO_FUNC_I2C) ;

// MPU6050 initialization
mpu6050_reset();
mpu6050_read_raw(acceleration, gyro);

// // ===== ADDED: seed filters with first reading so the angle doesn't start at 0 =====
filtered_ax = acceleration[0] ;
filtered_az = acceleration[2] ;
complementary_angle = multfix15(float2fix15(atan2(-filtered_ax, filtered_az)), oneeightyoverpi);
// // ===== END ADDED =====

// ===== ADDED: demo button input (must be set up before the ISR starts) =====
// Button wired GPIO -> button -> GND; internal pull-up makes released = 1.
gpio_init(BUTTON_PIN) ;
gpio_set_dir(BUTTON_PIN, GPIO_IN) ;
gpio_pull_up(BUTTON_PIN) ;
// ===== END ADDED =====

////////////////////////////////////////////////////////////////////////
////////////////////////// PWM CONFIGURATION ///////////////////////////
////////////////////////////////////////////////////////////////////////
// Tell GPIO's 4,5 that they allocated to the PWM
```

```
    gpio_set_function(5, GPIO_FUNC_PWM);
    gpio_set_function(PWM_OUT, GPIO_FUNC_PWM);  // CHANGED: was hardcoded 4, now uses PWM_OUT

    // Find out which PWM slice is connected to GPIO 5 (it's slice 2, same for 4)
    slice_num = pwm_gpio_to_slice_num(5);

    // Mask our slice's IRQ output into the PWM block's single interrupt line,
    // and register our interrupt handler
    pwm_clear_irq(slice_num);
    pwm_set_irq_enabled(slice_num, true);
    irq_set_exclusive_handler(PWM_IRQ_WRAP, on_pwm_wrap);
    irq_set_enabled(PWM_IRQ_WRAP, true);

    // This section configures the period of the PWM signals
    pwm_set_wrap(slice_num, WRAPVAL) ;
    pwm_set_clkdiv(slice_num, CLKDIV) ;

    // This sets duty cycle
    pwm_set_chan_level(slice_num, PWM_CHAN_B, 0);
    pwm_set_chan_level(slice_num, PWM_CHAN_A, 0);

    // Start the channel
    pwm_set_mask_enabled((1u << slice_num));


    //////////////////////////////////////////////////////////////////////
    ////////////////////////////// ROCK AND ROLL ////////////////////////////
    //////////////////////////////////////////////////////////////////////
    // start core 1
    multicore_reset_core1();
    multicore_launch_core1(core1_entry);

    // start core 0
    gpio_init(14) ;
    gpio_set_dir(14, GPIO_OUT) ;
    gpio_put(14, 0) ;
    pt_add_thread(protothread_serial) ;
    pt_schedule_start ;

}
```

## A.2 mpu6050.c (IMU driver)

```
/**
 * Hunter Adams (vha3@cornell.edu)
 * MPU6050 driver, used unchanged from the course demo (comments added for
 * the Lab 3 report). The MPU6050 is an I2C peripheral at address 0x68.
 * Every register access is: start, address + write, register number,
 * then either the data byte (write) or a repeated start, address + read,
 * and N data bytes (read). The register pointer auto-increments.
 */

#include "hardware/i2c.h"
#include "mpu6050.h"

// ----------------------------------------------------------------------
```

```
// mpu6050_reset(): wake the chip and configure it. Uses BLOCKING writes,
// so if SDA is stuck low this never returns (main() recovers the bus first).
//   0x6B PWR_MGMT_1   = 0x00 : leave sleep mode
//   0x19 SMPLRT_DIV   = 7    : 8 kHz / (1 + 7) = 1 kHz sample rate
//   0x1B GYRO_CONFIG  = 0x00 : +/-250 deg/s  (131 counts per deg/s)
//   0x1C ACCEL_CONFIG = 0x00 : +/-2 g        (16384 counts per g)
//   0x37 INT_PIN_CFG  = 0x10 : interrupt status clears on any read
//   0x38 INT_ENABLE   = 0x01 : data-ready interrupt (pin unused by Lab 3)
// ------------------------------------------------------------------------
void mpu6050_reset() {
    // Two byte reset. First byte register, second byte data
    // There are a load more options to set up the device in
    // different ways that could be added here
    uint8_t buf[] = {0x6B, 0x00};
    i2c_write_blocking(I2C_CHAN, ADDRESS, buf, 2, false);

    // Set gyro sample rate (set to 1KHz, same as accel)
    uint8_t gyro_rate[] = {0x19, 0b00000111} ;
    i2c_write_blocking(I2C_CHAN, ADDRESS, gyro_rate, 2, false);

    // Configure the Gyro range (+/- 250 deg/s)
    uint8_t gyro_settings[] = {0x1b, 0b00000000} ;
    i2c_write_blocking(I2C_CHAN, ADDRESS, gyro_settings, 2, false);

    // Configure the Accel range (+/- 2g's)
    uint8_t accel_settings[] = {0x1c, 0b00000000} ;
    i2c_write_blocking(I2C_CHAN, ADDRESS, accel_settings, 2, false);

    // Configure interrupt pin
    uint8_t pin_settings[] = {0x37, 0b00010000} ;
    i2c_write_blocking(I2C_CHAN, ADDRESS, pin_settings, 2, false);

    // Configure data ready interrupt
    uint8_t int_config[] = {0x38, 0x01} ;
    i2c_write_blocking(I2C_CHAN, ADDRESS, int_config, 2, false);
}

// ------------------------------------------------------------------------
// mpu6050_read_raw(): read all three accel axes (regs 0x3B-0x40) and all
// three gyro axes (regs 0x43-0x48), 6 bytes each, big-endian, and convert
// them to 15.16 fixed point:
//   accel: raw 16384/g, << 2 -> 65536/g = 1.0 g in 15.16
//   gyro:  raw 131 per deg/s, * 500 >> 16 = raw / 131.07 -> deg/s
// Called from the 1 kHz ISR. Each transfer has a 250 us timeout, so a
// missing ACK costs at most 250 us per call instead of hanging the ISR.
// About 0.21 ms of bus time per 6-byte read at 400 kHz, 0.42 ms total.
// ------------------------------------------------------------------------
void mpu6050_read_raw(fix15 accel[3], fix15 gyro[3]) {
    // For this particular device, we send the device the register we want to read
    // first, then subsequently read from the device. The register is auto incrementing
    // so we don't need to keep sending the register we want, just the first.

    uint8_t buffer[6];
    int16_t temp_accel, temp_gyro ;

    // Status value
```

```
    uint32_t success;

    // Start reading acceleration registers from register 0x3B for 6 bytes
    uint8_t val = 0x3B;
    i2c_write_timeout_us(I2C_CHAN, ADDRESS, &val, 1, true, 250); // true to keep master control of bus
    i2c_read_timeout_us(I2C_CHAN, ADDRESS, buffer, 6, false, 250);

    for (int i = 0; i < 3; i++) {
        temp_accel = (buffer[i<<1] << 8 | buffer[(i<<1) + 1]); // high byte first; int16_t keeps the sign
        accel[i] = temp_accel ;
        accel[i] <<= 2 ; // convert to g's (fixed point)
    }

    // Now gyro data from reg 0x43 for 6 bytes
    // The register is auto incrementing on each read
    val = 0x43;
    i2c_write_timeout_us(I2C_CHAN, ADDRESS, &val, 1, true, 250);
    i2c_read_timeout_us(I2C_CHAN, ADDRESS, buffer, 6, false, 250);  // False - finished with bus

    for (int i = 0; i < 3; i++) {
        temp_gyro = (buffer[i<<1] << 8 | buffer[(i<<1) + 1]);
        gyro[i] = temp_gyro ;
        gyro[i] = multfix15(gyro[i], 500<<16) ; // deg/sec
    }
}
```

## A.3 mpu6050.h (pins and fixed-point macros)

```
/**
 * Hunter Adams (vha3@cornell.edu)
 *
 *
 */

// I2C settings: MPU6050 at 0x68 (AD0 pin low) on I2C0, GPIO 8/9, 400 kHz
#define ADDRESS 0x68
#define I2C_CHAN i2c0
#define SDA_PIN  8
#define SCL_PIN  9
#define I2C_BAUD_RATE 400000

// Fixed point data type: signed 15.16 in a 32-bit int.
// Bit 31 sign, bits 30-16 integer, bits 15-0 fraction. 1.0 = 65536.
// multfix15 multiplies in 64 bits, then shifts the 32 fraction bits back to 16.
typedef signed int fix15 ;
#define multfix15(a,b) ((fix15)((( signed long long)(a))*(( signed long long)(b)))>>16))
#define float2fix15(a) ((fix15)((a)*65536.0f)) // 2^16
#define fix2float15(a) ((float)(a)/65536.0f)
#define int2fix15(a) ((a)<<16)
#define fix2int15(a) ((a)>>16)
// NOTE: divfix has a precedence bug: 16 / (b) is evaluated before the shift.
// Lab 3 never calls it; a correct form is ((fix15)((((signed long long)(a)) << 16) / (b))).
#define divfix(a,b) ((fix15)((( signed long long)(a) << 16 / (b))))
// Pre-scaled constants (value * 65536): 180/pi, 0.001 and 0.999 are used by
// the complementary filter; the others are spare weight options.
// Parameter values
```

```
#define oneeightyoverpi 3754936
#define zeropt001 65
#define zeropt999 65470
#define zeropt01 655
#define zeropt99 64880
#define zeropt1 6553
#define zeropt9 58982

// Driver functions (mpu6050.c)
void mpu6050_reset(void) ;
void mpu6050_read_raw(fix15 accel[3], fix15 gyro[3]) ;
```

### A.4 CMakeLists.txt

[TODO: replace with our CMakeLists.txt if it differs from the course demo's, shown here.]

```
# cmake version
cmake_minimum_required(VERSION 3.13)

# include the sdk.cmake file
include(pico_sdk_import.cmake)

# give the project a name (anything you want)
project(MPU6050_IMU C CXX ASM)

# initialize the sdk
pico_sdk_init()

add_executable(MPU6050_IMU)

# must match with pio filename and executable name from above
pico_generate_pio_header(MPU6050_IMU ${CMAKE_CURRENT_LIST_DIR}/VGA/hsync.pio)
pico_generate_pio_header(MPU6050_IMU ${CMAKE_CURRENT_LIST_DIR}/VGA/vsync.pio)
pico_generate_pio_header(MPU6050_IMU ${CMAKE_CURRENT_LIST_DIR}/VGA/rgb.pio)

# must match with executable name and source file names
target_sources(MPU6050_IMU PRIVATE imu_demo.c VGA/vga16_graphics_v3.c mpu6050.c)

# Add pico_multicore which is required for multicore functionality
target_link_libraries(  MPU6050_IMU
                        pico_stdlib
                        pico_bootsel_via_double_reset
                        pico_multicore
                        hardware_pwm
                        hardware_dma
                        hardware_irq
                        hardware_adc
                        hardware_pio
                        hardware_sync
                        hardware_i2c
                        hardware_clocks)

# create map/bin/hex file etc.
pico_add_extra_outputs(MPU6050_IMU)
```

## Appendix B: AI Prompt Log

The handout asks for the words exchanged, the number of code additions/deletions/modifications suggested, and the number accepted.

| Session and tool | Prompt (summary) | Words exchanged | Suggested (add / delete / modify) | Accepted (add / delete / modify) |
|---|---|---|---|---|
| [TODO: earlier coding sessions, tool name] | Build P, then D, then I control; complementary filter; I2C bus recovery; VGA plots (the code marked ADDED / CHANGED) | [TODO] | [TODO] | [TODO] |
| Claude, Oct 28 | "Here is our code so far": check it against the report and the handout | [TODO] | 7 / 0 / 3: demo button (defines and globals, GPIO setup, ISR state machine, VGA status line), low-passed display (motor_disp variable and ISR update), pin list in the header; plot buffer stores motor_disp; 2 stale comments fixed | [TODO] |
| Claude, Oct 28 | "Add more from the PDFs and answer the handout's questions": commented listing for the appendix | [TODO] | 9 / 0 / 2, all comments only: file overview, 6 function headers, 2 driver function headers and register notes, fixed-point notes in mpu6050.h; 2 comments reworded | [TODO] |

*Table 6: AI prompt log summary.*

[TODO: paste the full prompt log below. If the tool can generate it, ask it for "a prompt log with words exchanged and counts of code additions/deletions/modifications suggested and accepted."]

## References


1. V. H. Adams, PID control of a 1D helicopter (lab handout).
2. V. H. Adams, Complementary filters.
3. V. H. Adams, Phenomenological introduction to PID controllers.
4. V. H. Adams, Analytical introduction to PID controllers.
5. V. H. Adams, DC motor circuit and PWM.
6. V. H. Adams, Data display.
7. V. H. Adams, Inter-Integrated Circuit (I2C).
8. V. H. Adams, MPU6050 IMU demo code and PWM demo code.
9. B. Land, ECE 4760/5730 course policy (lab report sections).